# VoxWrite

AUTOMATIC SPEECH RECOGNITION
SMART DICTATION TOOL

# TASK 2 – ASR TOOL IMPLEMENTATION

Project Report

Department of Computer Science and Engineering
SRM Institute of Science and Technology
Tiruchirappalli

Student Name: _____
Register Number: _____
Class / Section: CSE _____

# 1. Abstract

VoxWrite is a browser-based Automatic Speech Recognition (ASR) smart dictation application. It captures spoken input through a microphone and converts it into editable text using the Web Speech API. The application supports continuous recognition, interim transcription, language selection, optional spoken-punctuation formatting, transcript editing, word and character counting, copying, and text-file export. An optional text-to-speech module reads the transcript aloud and demonstrates configurable voice styles.

# 2. Introduction

Speech recognition is a branch of artificial intelligence that enables computers to interpret human speech. ASR systems transform an audio signal into a sequence of words. They are used in dictation, virtual assistants, accessibility tools, call-centre automation, captioning, and voice-controlled applications. This project demonstrates the complete user-facing workflow of a small speech-to-text tool.

# 3. Problem Statement

Typing long passages can be time-consuming, and users may prefer speaking naturally to create notes or paragraphs. The project addresses this by providing a simple interface that captures speech, displays recognized words, and allows the user to edit and export the resulting transcript.

# 4. Objectives

• Implement a working browser-based speech-to-text interface.

• Capture live microphone speech and show interim and final recognition results.

• Preserve earlier transcript text when a new dictation session begins.

• Provide language selection and optional spoken-punctuation formatting.

• Support transcript editing, word/character counts, copy, and TXT download.

• Demonstrate text-to-speech playback and selectable voice-style presets.

• Document setup, use, testing, limitations, and GitHub submission steps.

# 5. Reference-Based Design

The uploaded ASR Studio report describes a larger architecture using a React/Node.js frontend, FastAPI backend, Faster-Whisper, Silero Voice Activity Detection, timestamped segments, and subtitle exporters. The uploaded Smart Dictation & Voice Style report describes a lightweight client-side design using HTML, CSS, JavaScript, the Web Speech API, a shared transcript box, punctuation formatting, text-to-speech, and voice presets.

VoxWrite adopts the second report's simpler browser-based implementation so that the project is easy to run and explain for a student task. It incorporates the shared transcript workflow, continuous/interim recognition, punctuation option, text management, TTS and voice-style controls. Unlike the larger ASR Studio, it does not claim offline inference, Whisper model execution, timestamp alignment, batch processing, or subtitle export.

# 6. System Architecture

The application follows a three-layer client-side architecture:

**Presentation Layer**
index.html + style.css
Controls, transcript editor, status, counters

**Application Logic Layer**
script.js
Recognition events, punctuation, transcript state, export and playback

**Browser Speech Services**
Web Speech API
SpeechRecognition and SpeechSynthesis

Data flow: User selects language → clicks Start speaking → browser requests microphone access → SpeechRecognition receives speech → interim/final results are raised → punctuation formatting is optionally applied → transcript editor is updated → user edits/copies/downloads the text. The TTS path reads the transcript and sends it to SpeechSynthesis for audio playback.

# 7. Modules

## 7.1 Speech Input and Recognition

The application checks for window.SpeechRecognition or window.webkitSpeechRecognition. It enables continuous recognition and interim results, sets the selected language, and listens for onstart, onresult, onerror, and onend events.

## 7.2 Punctuation Formatter

When enabled, the formatter replaces spoken cues such as “comma”, “period”, “full stop”, “question mark”, and “exclamation mark” with their corresponding punctuation marks.

## 7.3 Transcript Editor

The transcript is displayed in an editable text area. Word and character counts update whenever the user types or recognition changes the text. The user can copy the transcript, download it as a UTF-8 text file, or clear the editor.

## 7.4 Text-to-Speech and Voice Styles

SpeechSynthesisUtterance reads the transcript aloud. The normal mode uses standard pitch and rate. The optional style selector demonstrates Robot, Chipmunk, Narrator, Announcer, and Whisper presets by changing pitch/rate. With “Keep my words” unchecked, a simple sound cue is spoken for each word.

## 7.5 Error Handling

• Unsupported browser: show a compatibility message and disable recognition.

• Microphone permission denied: instruct the user to allow microphone access.

• No microphone: report audio-capture error.

• Network failure: advise checking connectivity; browser speech recognition may use an online service.

• No speech detected: ask the user to speak clearly and retry.

# 8. Technologies Used

| Technology | Purpose |
| --- | --- |
| HTML5 | Structure of the application and form controls |

| Technology | Purpose |
|---|---|
| CSS3 | Responsive layout, colours, spacing and visual design |
| JavaScript ES6+ | Recognition event handling, transcript state and UI actions |
| Web Speech API | SpeechRecognition for ASR and SpeechSynthesis for TTS |
| Chrome / Edge | Recommended browser runtime |
| VS Code + Live Server | Local development and HTTP serving |
| Git + GitHub | Version control and source-code submission |

## 9. Installation and Execution

• Extract the project ZIP or clone the GitHub repository.

• Open the folder in Visual Studio Code.

• Install and start the Live Server extension, then open index.html with Live Server.

• Use the local HTTP URL in Chrome or Edge.

• Allow microphone permission, select a language, and click Start speaking.

• Speak a sentence or paragraph, then click Stop. Edit, copy, or download the transcript.

• Optionally use Speak normally or choose a voice style for playback.

## 10. Testing and Quality Assurance

The source includes a manual testing checklist in README.md. Complete the tests on a supported browser and record the actual outcome before submission. The expected checks include microphone permission, live interim/final transcription, stopping and restarting without losing previous text, punctuation formatting, counter updates, copy/download, text-to-speech, and error messages.

| Test case | Expected result |
|---|---|
| Browser support check | Supported browser enables Start; unsupported browser shows a clear message. |
| Microphone dictation | Recognized words appear in the transcript area. |
| Punctuation cues | Spoken punctuation words are converted when option is enabled. |
| Transcript editing | User can type and edit; counts update. |
| Copy and download | Text is copied or saved as voxwrite-transcript.txt. |
| TTS playback | Text is read aloud; Stop audio cancels playback. |

## 11. Limitations and Future Enhancements

Recognition support and accuracy depend on the browser, language, accent, background noise, microphone quality, and network. Some browser implementations process speech through a vendor-operated service, so this prototype must not be described as fully offline or guaranteed private. Text-to-speech voices vary by operating system.

• Future work: add a Faster-Whisper backend for local model-based transcription.

• Add audio-file upload and subtitle export in SRT/VTT formats.

- Add timestamps, confidence information, and batch transcription.
- Improve multilingual punctuation and custom vocabulary support.
- Add automated browser tests and accessibility improvements.

## 12. Conclusion

VoxWrite demonstrates a practical ASR workflow in a compact web application. It captures spoken language, displays recognized text, supports transcript editing and export, and includes optional speech playback. The project satisfies the core learning objective of implementing and testing an ASR tool while clearly documenting browser-dependent behavior and future improvements.

## 13. GitHub Submission

Create a GitHub repository named voxwrite-asr-smart-dictation. Upload index.html, style.css, script.js, README.md, .gitignore, and this project report. After pushing the files, paste the actual repository URL into the college Google Drive submission folder. Do not submit a placeholder repository URL.